

IFN647 Week 5 review questions

SOLUTIONS

Q1 (Language Models)

Let $Q = \{\text{US, ECONOM, ESPIONAG}\}$ be a query, and $C = \{D_1, D_2, D_3, D_4, D_5, D_6\}$ be a collection of documents, where

$$\begin{aligned}D_1 &= \{\text{GERMAN, VW}\} \\D_2 &= \{\text{US, US, ECONOM, SPY}\} \\D_3 &= \{\text{US, BILL, ECONOM, ESPIONAG}\} \\D_4 &= \{\text{US, ECONOM, ESPIONAG, BILL}\} \\D_5 &= \{\text{GERMAN, MAN, VW, ESPIONAG}\} \\D_6 &= \{\text{GERMAN, GERMAN, MAN, VW, SPY}\}\end{aligned}$$

Jelinek-Mercer smoothing method uses the following equation to calculate a score for a document D

$$P(Q|D) = \prod_{i=1}^n p(q_i|D)$$

where n is the number of query terms in Q , and

$$p(q_i|D) = (1 - \lambda) \frac{f_{q_i,D}}{|D|} + \lambda \frac{c_{q_i}}{|C|}$$

for all query term q_i in Q , where $f_{q_i,D}$ is the number of times query word q_i occurs in document D , $|D|$ is the number of word occurrences in D , c_{q_i} is the number of times query word q_i occurs in collection C , and $|C|$ is the total number of word occurrences in collection C . Assume parameter $\lambda = 0.4$, calculate $P(Q|D_2)$. You are required to write the finding process.

Solution:

$$D_2 = \{\text{US, US, ECONOM, SPY}\}, Q = \{\text{US, ECONOM, ESPIONAG}\}$$

$$P(Q|D_2) = P(\text{US} | D_2) * P(\text{ECONOM} | D_2) * P(\text{ESPIONAG} | D_2) = 0.003865$$

$$P(\text{US} | D_2) = (1 - 0.4) * (2/4) + 0.4 * (4/23) = 0.6 * 0.5 + 0.4 * 0.17 = 0.368$$

$$P(\text{ECONOM} | D_2) = 0.6 * (1/4) + 0.4 * (3/23) = 0.6 * 0.25 + 0.4 * 0.13 = 0.202$$

$$P(\text{ESPIONAG} | D_2) = 0.6 * (0/4) + 0.4 * (3/23) = 0.4 * 0.13 = 0.052$$

Q2 (Likelihood Model)

Test the function $\text{likelihood_IR}(I, Q)$ introduced in the lecture notes using a Jupyter Notebook. First, implement the function $\text{likelihood_IR}(I, Q)$ as defined in the lecture notes. Then, apply this function to the data in the “Test_data” folder and a query $Q = \text{share bank}$. The output should be a ranked list of all documents in the folder, ordered by their likelihood scores with respect to the query Q .

Solution:

```
# define a function likelihood_IR(I,Q) to estimate P(Q|D) for all documents D in a folder indexed by I
def likelihood_IR(I, Q): #index I is a dirctionary of term:(itemId:freq)
    L={} # L is the selected inverted list
    R={} # R is a dirctionary of docId:score
    D_len={} # D_len is a dirctionary of docId:length
    for list in I.items():
        for id in list[1].items():
            R[id[0]]=1 # get all document IDs and initialize as 1
            D_len[id[0]]=0.5 # a small non-zero value as a denominator
        if (list[0] in Q): # select inverted lists based on the query Q
            L[list[0]]= I[list[0]]
    for q_term in Q.items(): # L may not contain all query terms
        if not(q_term[0] in L):
            L[q_term[0]]={}
    for list in I.items():
        for id in list[1].items(): #Count term occurrences in documents
            D_len[id[0]]= D_len[id[0]] + id[1]
    for (d, sd) in R.items():
        for (term, f) in L.items():
            if not(d in f):
                f[d]=0
            sd = sd*(f[d]/D_len[d]) #see the equation in the lecture notes
    R[d] = sd
    return R

# Test the defined functions
curr_path=os.getcwd()
stopwords_f = open('common-english-words.txt', 'r')
stop_words = stopwords_f.read().split(',')
stopwords_f.close()
Test_path = curr_path+'/Test_docs'
Index_Test = index_docs(Test_path, stop_words) #Index all terms
os.chdir(curr_path)
Query = {'share':1, 'bank':1}
#test the likelihood IR model
IR_result = likelihood_IR(Index_Test, Query)
y = sorted(IR_result.items(), key=lambda x: x[1],reverse=True)
print('Likelihood Query Result -----')
for (id, w) in y:
    print('Document ID: '+id + ' and relevance weight: ' + str(w))

Likelihood Query Result -----
Document ID: 807606 and relevance weight: 0.0009652315550282532
Document ID: 807600 and relevance weight: 0.00014154245894636803
Document ID: 809481 and relevance weight: 0.0
Document ID: 809495 and relevance weight: 0.0
```

Please note that you are required to call the function `index_docs()`, which was discussed in Module 3.

```
#Function index_docs() returns an index - a dictionary {term:{docID1:freq1, DocID2:freq2, ...}, ...}

import glob, os
import string
from stemming.porter2 import stem

def index_docs(inputpath,stop_words):
    Index = {} # initialize the index
    os.chdir(inputpath)
    for file_ in glob.glob("*.xml"):
        start_end = False
        for line in open(file_):
            line = line.strip()
            if(start_end == False):
                if line.startswith("<newsitem "):
                    for part in line.split():
                        if part.startswith("itemid="):
                            docid = part.split("=")[1].split("\"")[1]
                            break
                if line.startswith("<text>"):
                    start_end = True
            elif line.startswith("</text>"):
                break
            else:
                line = line.replace("<p>", "").replace("</p>", "")
                line = line.translate(str.maketrans('', '', string.digits)).translate(str.maketrans(string.punctuation, ' '*len(string.punctuation)))
                for term in line.split():
                    term = stem(term.lower())
                    if len(term) > 2 and term not in stop_words:
                        try:
                            Index[term][docid] += 1
                        except KeyError:
                            Index[term][docid]=1
                    except KeyError:
                        Index[term] = {docid:1}

    return Index
```

Q3 Which of the following are False? and justify your answer.

- (1) For a very large data collection, pooling technique is used to select top-k results from the rankings obtained by different retrieval algorithms. The results are merged into a pool, duplicates are removed, and the documents are presented in some random order to the people doing the relevance judgments.
- (2) Pooling produces a large number of relevance judgments for each query. However, this list is incomplete, and for a new retrieval algorithm that had not contributed documents to the original pool, this could potentially be a problem.
- (3) Many user actions (e.g., Query log data) can be considered implicit relevance judgments. The main drawback with this data is that it is not as precise as explicit relevance judgments.
- (4) A typical query log does not contain user identifier or user session identifier because of the privacy issue.

Solution: (4)

A typical query log contains user identifier or user session identifier, query terms, list of URLs of results, their ranks on the result list, and whether they were clicked on, and timestamp(s). The privacy is particularly an issue when query logs are shared, distributed for research, or used to construct user profiles.

Q4 (Effectiveness Metrics)

Assumption: Table 1 presents the relevance judgments for documents 110–124 with respect to topic 101, where a value of **1** indicates that the document is relevant and **0** indicates that it is non-relevant.

Table 2 reports the output of IR Model 1, in which documents are ranked in descending order according to their assigned weights.

Table 1. Relevant Judgements

TOPIC	DocNo	Rel
101	110	1
101	111	1
101	112	0
101	113	1
101	114	1
101	115	0
101	116	1
101	117	1
101	118	0
101	119	1
101	120	0
101	121	0
101	122	0
101	123	1
101	124	0

Table 2. IR Model 1's Output (ranked by weight)

TOPIC	DocNo	Weight
101	111	0.812
101	112	0.790
101	113	0.689
101	110	0.542
101	114	0.496
101	119	0.455
101	115	0.321
101	122	0.300
101	118	0.278
101	116	0.234
101	123	0.211
101	121	0.201
101	120	0.189
101	117	0.176
101	124	0.123

The set of
retrieved
documents

Assumption: Let A denote the set of relevant documents for topic 101, and A' denote the set of non-relevant documents. We further assume that IR Model 1 selects the top-6 ranked documents as the retrieved (relevant) set B (see Table 2), while B' represents the set of documents that are not retrieved.

- (1) List the elements of all sets (A , A' , B and B') and enclose them in braces.
- (2) Calculate recall and precision of IR model 1.
- (3) Calculate false positive and false negative of IR model 1.
- (4) Calculate the F_1 measure of IR model 1.

Solution:

(1)

$A = \{110, 111, 113, 114, 116, 117, 119, 123\}$

$B = \{111, 112, 113, 110, 114, 119\}$

$A' = \{112, 115, 118, 120, 121, 122, 124\}$

$B' = \{115, 122, 118, 116, 123, 121, 120, 117, 124\}$

(2)

$\text{Recall} = \frac{|A \cap B|}{|A|} = |\{110, 111, 113, 114, 119\}| / |A| = 5/8 = 62.5\%$

$\text{Precision} = \frac{|A \cap B|}{|B|} = |\{110, 111, 113, 114, 119\}| / |B| = 5/6 = 83.33\%$

(3)

$\text{FP} = \text{Fallout} = \frac{|A' \cap B|}{|A'|} = |\{112\}| / |A'| = 1/7$

$\text{FN} = \frac{|A \cap B'|}{|A|} = |\{116, 117, 123\}| / |A| = 3/8$

(4)

$\text{F1} = 2 * \text{Recall} * \text{Precision} / (\text{Recall} + \text{Precision}) = 2 * (5/8) * (5/6) / (5/8 + 5/6)$
 $= 10/14 = 71.43\%$